D

DOCKER

Complete Learning Notes

Explained in Simple, Understandable Language (Hinglish)

What's Inside

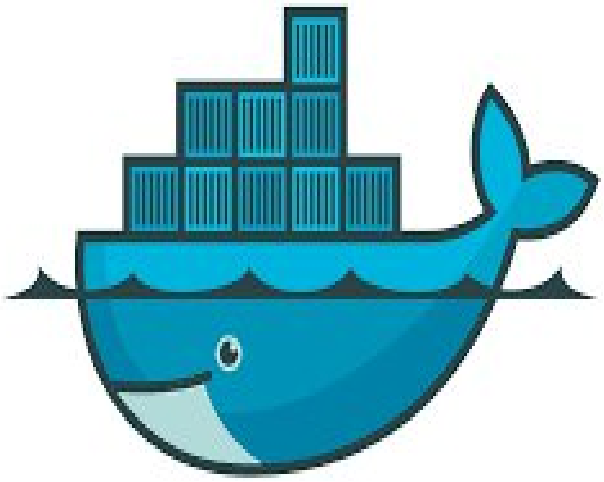
- Docker fundamentals — what it is and how it runs on Linux
- Images, containers, layers, and the Docker build/run lifecycle
- Dockerfiles, volumes, networking, and registries explained with analogies
- Real-world scenarios: Node.js apps, MySQL, multi-container setups
- Practical commands with step-by-step, easy-to-follow examples

NOTE

DOCKER

Very Easy and Understandable language in Hinglish language

Each topic Cover with Example



"We can install Docker on any OS, but the Docker engine runs natively on Linux distributions."

means the following:

1. Docker is available for all major operating systems:

You can **install Docker on Windows, macOS, and Linux**.

But behind the scenes, **Docker Engine** (which is the core component responsible for running containers) is designed to work **natively on Linux**. That's because:

- Docker containers use **Linux kernel features** like:
  - **cgroups** (control groups)
  - **namespaces**
  - **union file systems**
- These features are built into the **Linux kernel**.

2. What happens when you install Docker on Windows or macOS?

Docker still works, but:

- **Windows/macOS don't have a native Linux kernel**, so...
- Docker uses a **lightweight Linux virtual machine (VM)** behind the scenes.

Example:

- On **Windows**, Docker Desktop runs a Linux VM using **WSL 2** (Windows Subsystem for Linux v2) or **Hyper-V**.
- On **macOS**, Docker Desktop runs a Linux VM using **Apple HyperKit**.

So technically, **Docker containers are still running inside Linux**, but through a **virtualized environment**.

3. Docker on Linux (Native Support):

On Linux distros (Ubuntu, CentOS, Debian, etc.):

- Docker runs **directly on the host OS** using the Linux kernel.
- This means:
  - **Better performance** (no VM overhead)
  - **Simpler architecture**
  - **Fewer compatibility issues**

Summary:

OS	Docker Engine Runs	Uses VM?

	Directly?	
Linux	Yes	No
Windows	No	Yes (via WSL 2)
macOS	No	Yes (via VM)

In Simple Terms:

Docker *can be installed anywhere*, but it *truly belongs to Linux*.
On non-Linux systems, Docker needs a **Linux "helper" (VM)** to run properly.

Let me know if you'd like a diagram or command examples.

Question:-

explain in depely how docker work how container, image and all things are create and work in backend everythings in detail in undarstandable meaning and language

Answer:-

Sure, let's go step by step and understand **how Docker works** — including **images**, **containers**, and **what happens in the backend** — in **simple and deep detail**.

What is Docker?

Docker is a platform that allows you to **build, ship, and run applications in containers**.

Think of Docker like a lightweight virtual machine system that **packages your app + everything it needs** (code, libraries, dependencies) into one bundle: **a container**.

Core Concepts in Docker

Component	What It Is	Real-World Analogy
Image	A read-only template with app + dependencies	A blueprint or recipe
Container	A running instance of an image	A dish prepared from the recipe
Docker Engine	The core software that runs everything	The kitchen where cooking happens
Dockerfile	Script that tells Docker how to build the image	A written recipe
Registry	Storage for images (like Docker Hub)	A recipe book collection

How Docker Works (Step-by-Step)

Let's break this down:

1. Dockerfile – the instruction manual

You write a `Dockerfile`, which tells Docker:

- What base image to use (e.g., Ubuntu, Node.js, Python)
- What files to copy
- What commands to run (e.g., `apt install`, `pip install`)
- What port to expose
- What command to run when the container starts (e.g., `npm start`)

Example:

```
FROM node:18

WORKDIR /app

COPY . .

RUN npm install

CMD ["node", "app.js"]
```

This says:

- Start from Node 18
- Set working directory
- Copy files from your project
- Run `npm install`
- Then run `node app.js` when container starts

2. Docker builds the Image

When you run:

```
docker build -t myapp .
```

Docker does this:

1. Reads the `Dockerfile` line by line
2. Creates **layers** (more on this below)
3. Generates an **image** (like a snapshot of an OS + app)
4. Tags it with the name `myapp`

3. Docker Image – what it contains

A Docker image is:

- Read-only
- Built in **layers** (each instruction in Dockerfile is a layer)
- Uses **Union File System** (AUFS, OverlayFS)

Each layer is cached and reused. For example:

```
RUN apt install curl
```

This layer will only rebuild if the line changes.

Benefit: Fast builds and less storage usage.

4. Running the Image → Container

When you run:

```
docker run myapp
```

Docker:

1. Takes the image (`myapp`)
2. Creates a **writable layer on top**
3. Assigns:
    - Unique container ID
    - Network IP
    - Filesystem view (from image)
1. Runs the command in the image (`node app.js`)
2. The result is a **live container**

5. How Containers Work (Behind the Scenes)

Containers are not VMs. They share the **host OS kernel**, but are **isolated** using Linux features:

Linux Kernel Features Used by Docker:

Feature	What It Does
Namespaces	Isolate container processes (each container has its own PID, network, mounts, users)
cgroups (Control Groups)	Limit CPU, RAM, disk usage
Union File System	Merge layers (read-only image + writable container)

chroot / OverlayFS File system isolation

6. How Networking Works

Each container gets:

- Its own internal IP (via Docker's bridge network)
- You can expose ports from container to host using:
- `docker run -p 8080:80 myapp`
-

This maps:

- Host's port `8080` → Container's port `80`

7. Volumes – Saving Data

Containers are **temporary** — when stopped/deleted, data inside is lost.

To save data, Docker uses **volumes**:

```
docker run -v /host/data:/container/data myapp
```

This mounts a folder from host into the container — useful for databases, logs, uploads, etc.

8. Docker Registry (like Docker Hub)

You can **push and pull images**:

- `docker push myuser/myapp` → uploads to Docker Hub
- `docker pull node:18` → downloads Node image

You can host your own private registry too.

Summary Diagram (Visual in Text)

```
Dockerfile --> Docker Build --> Image (multi-layered, read-only)

Image + Container Runtime --> Container (live, isolated app)


Container = Image + writable layer
```

Lifecycle of a Docker Workflow

```
# 1. Write Dockerfile
```

```
# 2. Build image
docker build -t myapp .

# 3. Run container
docker run -d -p 8080:80 myapp

# 4. Save/share
docker tag myapp username/myapp
docker push username/myapp

# 5. Pull and run elsewhere
docker pull username/myapp
docker run -p 8080:80 username/myapp
```

🔍 Quick Recap:

Term	Purpose
`Dockerfile`	Instructions to build an image
`Image`	Blueprint of app (read-only)
`Container`	Running instance of an image
`Volume`	Persistent storage
`Network`	Isolated virtual network for containers
`Registry`	Image storage (like GitHub for Docker)

Would you like a visual diagram or a working example project next?

Great! Let’s take **real-world scenarios** to explain how Docker works — so you can truly understand **images, containers, Dockerfiles, networks, volumes**, and **what happens in the backend**.

Scenario 1: Hosting a Node.js Web App with Docker

Goal:

You made a simple Node.js app (e.g., a To-Do list). You want to run it in Docker so it works the same on any machine (your laptop, cloud, etc.).

Step-by-Step:

Step 1: Create the App

app.js

```
const express = require('express');

const app = express();

app.get('/', (req, res) => res.send('Hello from Docker!'));

app.listen(3000, () => console.log('Server running on port 3000'));
```

package.json

```
{
  "name": "docker-demo",
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

Step 2: Write the Dockerfile

```
FROM node:18           # Base OS + Node

WORKDIR /app           # Directory inside container

COPY . .               # Copy app files

RUN npm install        # Install dependencies

CMD ["node", "app.js"] # Start the app
```

Step 3: Build the Docker Image

```
docker build -t node-app .
```

What happens in backend:

- Docker reads `Dockerfile` line by line
- Creates layers:
  - Node base OS (from Docker Hub)
  - App code
  - Installed packages
- Produces a final **image** named `node-app`

Step 4: Run a Container

```
docker run -d -p 3000:3000 node-app
```

Behind the scenes:

- Docker takes the image
- Adds a writable layer (for temp data)
- Allocates a container ID
- Assigns internal IP, connects to default bridge network
- Maps host port 3000 → container port 3000
- Runs `node app.js`
- Your app is now live on `http://localhost:3000`

Real-World Analogy:

Docker Concept	Real-World
Dockerfile	Recipe
Image	Frozen food packet
Container	Cooked dish
Docker Engine	Kitchen
Registry	Supermarket
`docker run`	Cooking the food

Scenario 2: Saving Data from a MySQL Container

Goal:

Run a MySQL database using Docker, but **don't lose the data** when the container stops.

Step 1: Run MySQL with Volume

```
docker run -d \
  --name mysql-db \
  -e MYSQL_ROOT_PASSWORD=rootpass \
  -v /mydata:/var/lib/mysql \
  mysql:8
```

Behind the scenes:

- Docker pulls `mysql:8` image from Docker Hub
- Starts container named `mysql-db`
- Sets root password via env variable
- Mounts host folder `/mydata` into container's `/var/lib/mysql` (where MySQL stores data)

So even if the container is deleted:

```
docker rm -f mysql-db
```

You can run again, and data is still there in `/mydata`.

Analogy:

Concept	Real-World
Volume	External hard drive
Container	Temporary computer
Deleting container	Powering off computer
Data persists?	Yes, because stored outside container

Scenario 3: Two Apps Talking via Docker Network

Goal:

Run a **backend Node.js app** and a **frontend React app**, both in containers, and let them talk.

Step 1: Create Docker Network

```
docker network create mynet
```

Step 2: Run Backend API

```
docker run -d --name backend --network mynet backend-image
```

Step 3: Run Frontend App

```
docker run -d --name frontend --network mynet frontend-image
```

In frontend code, use:

```
fetch('http://backend:3000/api/data')  // 'backend' is container name
```

Behind the scenes:

- Both containers are on same network (`mynet`)
- Docker provides internal DNS
- So `backend` can be resolved by name

Analogy:

Concept	Real-World
Docker network	Office network
Containers	Computers
DNS	Name directory
Communication	Apps talking internally via IP or name

Summary

Action	Backend Mechanism
Build image	Layered file system (AUFS/OverlayFS)
Run container	Uses cgroups + namespaces for isolation
Port mapping	NAT: host-to-container IP/port translation
Volume	Bind mount or named volume on host
Network between containers	Docker bridge network + DNS resolution

Would you like me to create a working GitHub project example or draw a diagram of this architecture?

Question :-

secario in front of you a interviewer ask a question that how to automate 3-Tir application in AWS using docker, kubernetics, ci/cd, and other tools explain me each and every important point in detail

Answer:-

let's walk through the full **scenario of automating a 3-tier application on AWS** using **Docker**, **Kubernetes**, **CI/CD**, and supporting DevOps tools — in a way that you can confidently explain this to an interviewer.

What is a 3-Tier Application?

It's an app divided into **three separate layers**:

Tier	Description	Example Technologies
Frontend (UI)	User interface	React, Angular, HTML/CSS
Backend (API)	Business logic & processing	Node.js, Java, Python
Database	Data storage	MySQL, PostgreSQL

Goal of the Interview Question:

"How will you automate deployment and scaling of a 3-tier app in AWS using Docker, Kubernetes, and CI/CD?"

That means:

- Containerize it
- Deploy it on cloud (AWS)
- Automate everything with CI/CD
- Secure, monitor, and scale it

Your Answer Structure (Professional Format)

You can follow this **step-by-step flow** when answering:

1. Architecture Planning

- Identify 3 components:
  - UI (React frontend)
  - API (Node.js backend)
  - Database (MySQL/PostgreSQL)

Design:

```
Users → Load Balancer → Frontend Pod → Backend Pod → DB Pod (PVC attached)
```

AWS Infrastructure:

- **EKS** (Elastic Kubernetes Service) for container orchestration
- **ECR** (Elastic Container Registry) for storing Docker images
- **RDS** for database (or run MySQL in container with volume)

2. Dockerize All Components

Each tier needs its own **Dockerfile**.

Example: Backend Dockerfile (Node.js)

```
FROM node:18

WORKDIR /app

COPY . .

RUN npm install

CMD ["node", "server.js"]
```

Then:

```
docker build -t backend .
```

Push to AWS ECR:

```
aws ecr create-repository --repository-name backend

docker tag backend <account-id>.dkr.ecr.region.amazonaws.com/backend

docker push <account-id>.dkr.ecr.region.amazonaws.com/backend
```

Repeat for **frontend** and **database**.

3. CI/CD Pipeline Setup (Jenkins or GitHub Actions)

Tools used:

- Jenkins or GitHub Actions
- SonarQube (code quality)
- Docker
- Trivy (image scanning)
- Kubernetes (deployment)

CI/CD Flow:

```
[CI]

1. Developer commits code to GitHub

2. Jenkins pipeline is triggered

3. Lint & test code

4. Scan code with SonarQube

5. Build Docker images

6. Scan image with Trivy

7. Push image to ECR


[CD]

8. Pull latest image from ECR

9. Deploy to EKS using kubectl or Helm

10. Notify via Slack or email
```

4. Kubernetes Deployment on AWS (EKS)

Use:

- `Deployment` for frontend and backend pods
- `Service` for internal/external access
- `ConfigMap` for environment configs

- `Secret` for DB credentials
- `PersistentVolumeClaim` for DB storage

Example: backend-deployment.yaml

```
apiVersion: apps/v1

kind: Deployment

metadata:

  name: backend

spec:

  replicas: 2

  selector:

    matchLabels:

      app: backend

  template:

    metadata:

      labels:

        app: backend

    spec:

      containers:

      - name: backend

        image: <ECR-image-url>

        ports:

        - containerPort: 3000

---

apiVersion: v1

kind: Service

metadata:

  name: backend-svc

spec:
```

```
type: ClusterIP

selector:

  app: backend

ports:

  - port: 3000

    targetPort: 3000
```

Do the same for frontend and DB.

5. Database Setup

Two options:

Option A: Use AWS RDS (recommended)

- Create MySQL/PostgreSQL via AWS Console
- Store credentials in **Kubernetes Secrets**
- Backend connects using env variables

Option B: Deploy MySQL Pod

- Use persistent volume for data storage

6. Load Balancing and Ingress

- Use **AWS ALB Ingress Controller**
- Define ingress rules:
- `/ → frontend`
- `/api → backend`
-

Ingress lets external users access your app securely.

7. Monitoring and Logging

- **Prometheus + Grafana** for metrics (CPU, memory)
- **ELK Stack** or **AWS CloudWatch** for logs
- **Loki** (optional for lightweight log aggregation)

8. Security

- Use **Trivy** to scan Docker images

- Use **IAM roles** properly for Kubernetes and Jenkins
- Use **Kubernetes Secrets** for passwords/API keys
- Enable **HTTPS** on Ingress with TLS

9. Scaling and Auto-Healing

- Define **replicaSets** in deployments
- Set up **Horizontal Pod Autoscaler**
- AWS EKS handles node scaling with **Cluster Autoscaler**

Example Answer (What You Can Say to Interviewer):

“I would containerize each tier of the application using Docker — the frontend, backend, and database. Then, I’d push the Docker images to AWS ECR. I’d set up a Jenkins pipeline to automate CI steps like testing, SonarQube scanning, image build, and push to ECR.

For deployment, I’d use AWS EKS, writing Kubernetes manifests for each component — deployments, services, and ingress. I’d also connect to an RDS instance for database storage and manage secrets using Kubernetes Secrets. To scale, I’d configure Horizontal Pod Autoscaler and set up monitoring using Prometheus and Grafana. All this ensures a fully automated, secure, and scalable architecture.”

Bonus Tools You Can Mention:

Task	Tool
CI/CD	Jenkins, GitHub Actions
Code quality	SonarQube
Image scanning	Trivy
Monitoring	Prometheus, Grafana
Logging	ELK or CloudWatch
Secrets	Kubernetes Secrets
Deployment mgmt	Helm or ArgoCD (optional)

Would you like me to give you the actual Jenkins pipeline code and Kubernetes YAMLs to try this hands-on?

Question:-

Why do we need containers?

Answer:-

Let’s break this down **step by step** in **simple language**, with **real-world analogies** — so you truly understand:

Why Do We Need Containers?

Because running software is hard when:

- It behaves differently on **your laptop** vs **production**
- It needs **different versions** of Python, Node.js, Java, etc.
- It depends on **other services** like databases
- Setting it up takes time for every developer, server, or cloud

What Containers Solve:

1. "It works on my machine" problem

Developers often say:
“It runs fine on my laptop, but not on the server!”

Containers fix this by **packing the app + dependencies + environment** into one isolated unit.

2. Consistent across all environments

With containers, your app runs the **same** on:

- Developer laptop
- Test server
- Production server
- Cloud (AWS, GCP, Azure)

Same container = same behavior, everywhere

3. Faster deployment

No need to install packages manually on servers.

Just pull a container image and run it in seconds:

```
docker run my-app
```

4. Isolated and lightweight

Each container has its **own environment** and doesn't affect others.

You can run:

- Python app
- Java app
- Node.js app
  on same server — no conflicts.

5. Easy scaling

Want to serve more users?

Just run more containers:

```
docker run -d my-app
```

```
docker run -d my-app
```

6. Works well with DevOps tools

Containers are:

- Easy to test
- Easy to deploy in CI/CD
- Easy to run in **Kubernetes** or **AWS ECS**

Real-World Analogy

Think of a **container like a lunchbox**

Lunchbox (Container)	Real Meaning
Food (your app)	Your code
Side dishes (deps)	Libraries, configs
Box	Isolated runtime
Microwave safe	Runs anywhere

You can carry your lunchbox to:

- Home
- Office
- Picnic
  And your food always stays the same.

Just like your app in a container!

Without Container vs With Container

Without Containers	With Containers
Manual setup (e.g., apt, pip)	One image with everything ready
OS-level conflicts	App runs isolated
Slower deployment	Instant start
Works only on some machines	Works everywhere

Summary

Benefit	Why It Matters
Consistency	Same app behavior across systems
Portability	Run on laptop, server, or cloud
Isolation	No version conflicts
Speed	Start in seconds
DevOps-ready	Perfect for CI/CD and Kubernetes

Would you like to see a **live example of running a container** to try it yourself?

Question:-

what is namespace in docker explain in very easy language ?

Answer:-

Awesome, ! Let's go step-by-step and **see namespaces in action** with real `docker run` commands and how they isolate **processes, network, and files** in containers.

Prerequisites:

Make sure Docker is installed and running on your system. If not, let me know — I'll help you set it up.

Step-by-Step: Testing Namespaces in Docker

Step 1: Run Two Containers

Let's run two Ubuntu containers in the background:

```
docker run -dit --name container1 ubuntu bash
```

```
docker run -dit --name container2 ubuntu bash
```

- `-d`: detached mode (runs in background)
- `-i`: interactive
- `-t`: terminal
- `bash`: run bash shell

Step 2: View Process List Inside Each Container

Now enter each container and list processes:

```
docker exec -it container1 bash
```

```
ps aux
```

Output might show:

```
USER   PID  %CPU COMMAND
```

```
root     1  0.0  bash
```

```
root    10  0.0  ps aux
```

Now open another terminal and run:

```
docker exec -it container2 bash
```

```
ps aux
```

Even though both are running, **container1 can only see its own processes**, not those in container2.

This is PID namespace isolation.

Step 3: Check Network Inside Each Container

Inside container1:

```
docker exec -it container1 bash
```

```
ip a
```

You'll see a private IP like `172.17.0.2`.

Now inside container2:

```
docker exec -it container2 bash
```

```
ip a
```

It may show `172.17.0.3`.

These are **separate network namespaces** — each container has its **own virtual network interface**.

Step 4: Try File System Isolation

Inside container1:

```
docker exec -it container1 bash
```

```
ls /
```

You'll see a Linux root file system, but it's isolated. Try creating a file:

```
touch /test-in-c1
```

Now check container2:

```
docker exec -it container2 bash
```

```
ls /
```

You won't see `test-in-c1`.

This shows **mount namespace**: each container has its **own file system view**.

Summary of What You Just Saw

Test	Namespace Type	What Happened
`ps aux`	**PID**	Containers have own processes
`ip a`	**NET**	Containers have own IP addresses
`touch` + `ls`	**MNT**	Containers have their own file

system

Bonus: See Namespace IDs Directly

You can check namespace IDs with:

```
docker inspect container1 | grep -i sandbox
```

Or from host using `lsns` (Linux tool to view namespaces):

```
sudo apt install util-linux
lsns
```

It shows each container in its own namespace (pid, net, etc).

Would you like to explore **how to break out of a namespace** (just for learning) or how **Kubernetes also uses namespaces but in a different way**?

Question:-

also explain Cgroups in easy language

ANswer:-

After understanding **namespaces**, now let's understand **cgroups** — another core part of how Docker containers work under the hood.

What is a Cgroup? (Very Simple Language)

Cgroup = Control Group

☞ It is a Linux kernel feature that allows Docker to **control and limit**:

Resource	Example
CPU usage	Limit container to 1 core
Memory usage	Limit container to 512MB

Disk I/O	Limit how fast it reads/writes
Network bandwidth	(Advanced setup)

Why Are Cgroups Important?

Because containers run on the same host, and without limits:

- One container could use **all the CPU or RAM**
- It can crash the system or slow other containers

Cgroups prevent this by **setting boundaries** around each container's resource usage.

Real-Life Analogy

Imagine your computer is a **school canteen**, and containers are **students**.

- Namespaces = Each student has their own **plate** — their own private world.
- Cgroups = Each student can take only **one scoop** of rice, not the whole pot

So, everyone gets their fair share and doesn't starve or overload the system.

Example: Using Cgroups in Docker

You don't use "cgroups" directly — Docker handles it. But you **use flags** to apply limits.

Limit CPU usage

```
docker run -dit --name myapp --cpus="1" ubuntu
```

This container can only use 1 CPU core, no matter what.

Limit Memory usage

```
docker run -dit --name myapp --memory="512m" ubuntu
```

Max 512MB RAM. If the app tries to use more, it crashes (OOM error).

Combine CPU + Memory Limits

```
docker run -dit --name myapp --cpus="0.5" --memory="256m" ubuntu
```

Half a CPU + 256MB RAM

🔍 How to See Cgroups in Action

Step 1: Run a limited container

```
docker run -dit --name limited --cpus=0.5 --memory=256m ubuntu
```

Step 2: Go inside and stress it (optional)

Install stress tool:

```
apt update && apt install stress
```

Run:

```
stress --cpu 2 --vm 1 --vm-bytes 300M --timeout 30s
```

You'll see it gets **killed or throttled** because cgroups prevent over-usage.

In Short:

Feature	Namespace Does...	Cgroup Does...
Isolation	Gives containers separate space	Prevents them from taking too much
Example	"Each in its own room"	"Each has a food quota"

Summary

Cgroups (Control Groups) allow Docker to **limit and monitor resource usage** like CPU, memory, disk I/O.

They **protect the host system** by making sure one container doesn't hog everything.

Would you like a **visual diagram** to show how Cgroups and Namespaces work together in Docker?

Question:-

also explain Union FileSystem

Answer:-

What is a Union File System (UnionFS) in Docker?

In Simple Language:

A **Union File System** is a special type of file system that lets Docker **stack multiple layers of files** into a single view — like a **layered sandwich** .

It **combines multiple file layers into one virtual filesystem** — so your app sees just one folder structure.

Why Docker Uses UnionFS?

Docker uses UnionFS because it allows:

Benefit	Explanation
Image layering	Base OS + app code + dependencies = layered image
Reusability	Share common base layers across images
Faster builds	Only changed layers are rebuilt
Low storage usage	Saves only what has changed

Real-World Analogy

Think of your container like a **stack of transparent sheets**:

- Layer 1: Ubuntu base image (common for all)
- Layer 2: Install Node.js
- Layer 3: Copy app code
- Layer 4: Install `npm` packages
- Layer 5: Run your app

🔍 You see **one complete view**, but Docker knows it's made of layers.

How Docker Uses UnionFS Internally

When you run a container:

1. Docker combines **read-only layers** from the image (OS, app code)
2. Adds a **top writable layer** for the container to make changes
3. All layers are merged using UnionFS

You see a normal file system inside the container:

```
docker exec -it mycontainer bash
```

```
ls /
```

But behind the scenes:

```
- Layer 1: Ubuntu (read-only)
```

```
- Layer 2: Node.js installed (read-only)
```

```
- Layer 3: App files (read-only)
```

```
- Layer 4: Writable layer (container's changes)
```

Example: Image Reuse

Imagine you build two images:

- `Image A` (Ubuntu + Python)
- `Image B` (Ubuntu + Node.js)

Docker stores the **Ubuntu base layer only once**, and reuses it for both — saving time and disk space!

UnionFS Implementations Used by Docker

Docker can use different **UnionFS drivers**, depending on OS and settings:

Driver Name	Description
`overlay2`	Most modern, efficient driver
`aufs`	Older, used in early Docker
`btrfs`	B-tree file system
`zfs`	Advanced file system (less common)

Most Linux systems now use `overlay2` by default.

Summary

Term	Meaning
UnionFS	Combines multiple layers into one view

Question:-

Docker Engine in detail

Answer:-

Docker Engine kya hota hai, kaise kaam karta hai, aur industry mein iski kya importance hai — sab kuch **simple Hindi-English mix** mein.

Docker Engine Kya Hota Hai?

Docker Engine ek **client-server application** hai jo aapke system par **containers run karne** ke liye responsible hota hai.

Components of Docker Engine

Docker Engine mainly 3 parts mein divided hota hai:

Component	Role / Explanation
🖳 **Docker CLI (Client)**	Jo command aap terminal mein dete ho, jaise `docker run`
⚙ **Docker Daemon (dockerd)**	Background mein chalta hai, containers manage karta hai
REST API	Client aur Daemon ke beech communication ka bridge

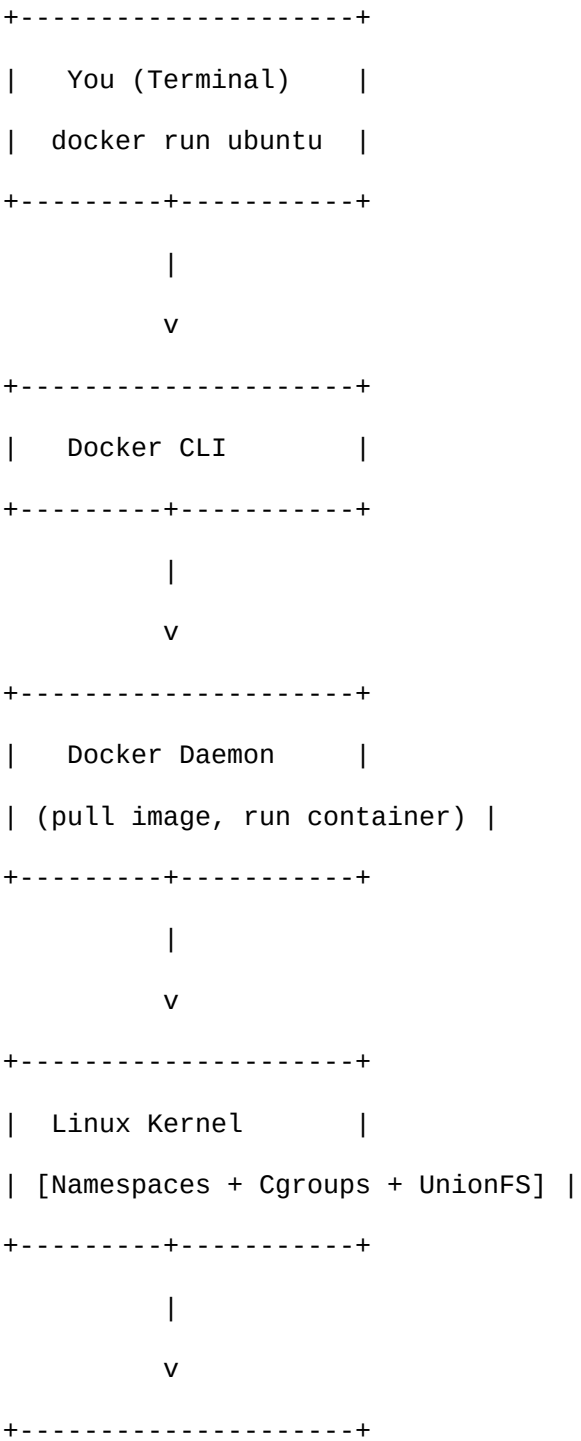
How Docker Engine Works (Simple Flow)

1. **Aap command dete ho:**
2. `docker run ubuntu`
3.
4. Ye command Docker Client ko jati hai.
5. **Docker Client request bhejta hai Docker Daemon ko** via REST API.
6. **Docker Daemon**:
    - Image pull karta hai (if not available)

- Container create karta hai
- Namespace + Cgroup + UnionFS set karta hai
- Container ko start karta hai

1. **Container run hota hai**, and aapke app ka isolated environment ready ho jata hai.

Graphical Overview:



```
|   Running Container |
+---------------------+
```

🔍 What Happens Internally When You Run a Container

Example: `docker run ubuntu`

1. **Docker Image pull** hoti hai from Docker Hub (if not cached)
2. Docker ne:
    - UnionFS se layers banayi (Ubuntu layer + writable layer)
    - Namespace create kiya (PID, NET, MNT, etc.)
    - Cgroups se memory/CPU limits apply kiya
1. Container chalu hota hai
2. Aap `docker ps` se dekh sakte ho running containers

Where is Docker Engine installed?

Platform	Docker Engine Setup
Linux	Direct install of Docker Daemon
Windows/Mac	Docker Desktop (jo internally Linux VM run karta hai)

📦 Docker Engine vs Docker Desktop?

Feature	Docker Engine (Linux)	Docker Desktop (Win/Mac)
Native performance	Yes	× Slower due to VM
Background daemon	`dockerd` directly	Runs in internal VM
Use case	Production, servers	Development, learning

⚙ Systemd Status Commands

Check daemon running:

```
sudo systemctl status docker
```

Start docker daemon:

```
sudo systemctl start docker
```

Summary in One Line:

Docker Engine is the **heart** of Docker — it receives your commands, creates isolated containers, manages images, and makes everything run smoothly using Linux kernel features.

Question:-

Data Persistance kya hota hai

Answer:-

Docker mein Data Persistence kya hota hai?

Chalo isse **simple words** aur **real examples** ke saath samjhte hain.

📦 What is Data Persistence in Docker?

Persistence ka matlab hota hai:

"**Data ko container ke baahar bhi save karna**, taaki container delete ho jaye phir bhi data na delete ho."

❌ By default:

Agar aap ek container mein file banao aur phir us container ko delete kar do:

```
docker run -it ubuntu
touch hello.txt
exit
docker rm <container_id>
```

➡ File `hello.txt` bhi chali jaayegi ❌
Yani Docker container ka data temporary hota hai.

Example Situation:

Socho aap ek **MySQL container** run kar rahe ho:

```
docker run -d --name mysql -e MYSQL_ROOT_PASSWORD=pass mysql
```

Ab agar aapka container crash ya delete ho gaya ✕ to:

- Saari database files gayab
- Saara user ka data chala gaya

Solution: Use Volumes or Bind Mounts

Docker ke paas 2 main ways hain **data ko persistent** banane ke liye:

1. Docker Volume (Recommended)

```
docker volume create mydata
```

```
docker run -d --name mysql -v mydata:/var/lib/mysql -e
MYSQL_ROOT_PASSWORD=pass mysql
```

🟢 Ab data `mydata` naam ke volume mein safe rahega, even after container is deleted.

2. Bind Mount (Host Folder Use)

```
docker run -d --name mysql -v /home/suraj/mysql-data:/var/lib/mysql -e
MYSQL_ROOT_PASSWORD=pass mysql
```

🟢 Isme Docker host (your Linux system) ke folder ka use hota hai — data wahaan save hota hai.

📊 Compare: Volume vs Bind Mount

Feature	Volume	Bind Mount
Setup	Docker manages it	You specify host path
Portability	Portable	Depends on host path
Permissions	Docker handles	You manage manually
Performance	Better for Docker storage	Slightly slower sometimes

Real-Life Analogy

Socho container ek **temporary kitchen** hai
Aap khana bana rahe ho. Lekin:

- Agar kitchen jala diya (delete container), to **sab khana waste**.
- Agar fridge (volume) mein khana store kiya, to **kitchen jale to bhi khana safe**.

Fridge = Volume = Persistent Storage

Summary:

Term	Meaning
Data Persistence	Container delete hone ke baad bhi data safe
Without it	Data chala jaata hai with container
With Volumes	Data alag store hota hai, safe rehta hai
Real Use	Databases, uploads, logs, user files etc.

Commands Recap:

Create volume:

```
docker volume create myvol
```

Use in container:

```
docker run -v myvol:/app/data ubuntu
```

See volumes:

```
docker volume ls
```

Inspect volume:

```
docker volume inspect myvol
```

Question:-

What is Ephemeral FS in detail

Answer:-

Ephemeral File System (Ephemeral Storage) in Docker

What is "Ephemeral"?

"**Ephemeral**" ka matlab hota hai **temporary** — yani **kuch samay ke liye**, jo **permanent nahi hota**.

So, What is Ephemeral File System in Docker?

Docker container ke andar jo file system hota hai — wo by default **ephemeral** hota hai.

Matlab:

- Jab tak container chal raha hai, tab tak files available hain
- Lekin **container delete hone par saari files delete ho jaati hain** ×

📦 Real-World Example:

```
docker run -it ubuntu
```

Inside container:

```
cd /app

mkdir test

echo "Suraj DevOps Engineer" > test/info.txt

exit
```

Ab agar aap container ko delete kar do:

```
docker rm <container_id>
```

➡ `/app/test/info.txt` bhi **delete ho jaayega permanently** 😔

Yahi hai ephemeral storage behavior.

Ephemeral vs Persistent Storage

Feature	Ephemeral Storage	Persistent Storage (Volumes)
Lifetime	Container ke lifetime tak	Container ke baahar bhi safe
Deleted on container rm	Yes	× No
Real-world example	Logs/temp files during runtime	Database files, user uploads
Use-case	Cache, temporary work	Permanent important data

Why Docker Uses Ephemeral FS by Default?

Because:

- Lightweight hota hai
- Fast hota hai
- Containers stateless banaye jaate hain (run & forget model)

Agar aapko **stateful / persistent data** chahiye, to aapko explicitly:

- `volume` ya
- `bind mount` use karna padta hai.

Inside Docker: Writable Layer is Ephemeral

Docker image = multiple read-only layers (UnionFS)
Container = Image + top **writable layer**

Wahi writable layer ephemeral hoti hai — jab container delete hota hai, wo bhi delete ho jaata hai.

Summary (Easy Words)

Question	Answer
Ephemeral FS kya hai?	Temporary file system inside container
Kab delete hota hai?	Jab container delete hota hai ya stop ke baad
Kya data safe rahta hai?	× No, sab kuch chala jaata hai

Solution kya hai? Volumes ya bind mounts use karo for safety

Real-Life Analogy:

Socho aap ek **whiteboard** par likh rahe ho (container ke andar):

- Jab tak board hai, likha dikh raha hai
- Jaise hi board hata diya (container delete), likha bhi chala gaya ×

Agar likha safe rakhna hai, to notebook (volume) mein likhna padega

Question:-

Inthis using 3rd party container image meaning

Answer:-

"Using third-party container image" ka matlab industry mein bahut important hota hai —

Third-Party Container Image Kya Hota Hai?

Jab hum **kisi aur developer ya company dwara banaya hua Docker image** use karte hain (jo humne khud nahi banaya), usse **third-party image** kehte hain.

Ye image mostly Docker Hub ya kisi aur container registry (like GitHub Container Registry, Quay.io, etc.) pe available hoti hai.

📦 Real-World Examples:

Image Name	Creator	Use Case
`mysql`	Oracle	Database image
`nginx`	NGINX, Inc.	Web server
`redis`	Redis maintainers	Caching server
`node`, `python`	Docker Official	Runtime for Node.js or Python apps

`bitnami/mongodb`	Bitnami (3rd party)	MongoDB with added configs

Example:

You want to run a MySQL database.

```
docker run -d --name db -e MYSQL_ROOT_PASSWORD=pass mysql
```

👉 Yahan `mysql` image aapne Docker Hub se pull kiya hai — ye **official image** hai.

Lekin agar aap use karte ho:

```
docker run bitnami/mysql
```

🟢 To aap **third-party vendor (Bitnami)** ka banaya hua image use kar rahe ho.

Why Use Third-Party Images?

Reason	Example
Save time	Ready-to-use prebuilt images
Popular configurations	Bitnami images pre-configured with best practices
Tools not available officially	Like SonarQube, Jenkins with plugins
Community-maintained	Many people contribute to make it better

⚠ Risks of Using Third-Party Images

Risk	Why It Matters
Security issues	Unknown code could have vulnerabilities
Hidden malware/backdoors	If image is not trusted
× Wrong functionality	Could behave unexpectedly
⚠ Licensing issues	May not be open-source

How to Use Safely

1. **Always check image source**
   Use verified accounts or official sources
2. **Use image digests or tags carefully**
   Avoid `latest`, prefer versioned tags
   e.g., `bitnami/mysql:8.0.36-debian-11-r0`

3. **Scan images** using tools like:
    - Trivy
    - Clair
    - Docker Scout (built-in)
1. **Read Docker Hub page** for:
    - Dockerfile used
    - Exposed ports
    - Environment variables

Example: Check who made the image

```
docker pull nginx
```

This pulls from official nginx Docker Hub page

But:

```
docker pull suraj/nginx
```

Yeh kisi unknown person ne banaya ho sakta hai. Ye image third-party hai — verify karna zaroori hai.

Summary

Term	Meaning
Third-Party Image	Docker image jo aapne khud nahi banayi
Use	Prebuilt tools, services, databases
Source	Docker Hub, GitHub Container Registry, etc.
Check	Trust, security, source, reviews

Question:-

Building Container Images deeply explain

Answer:-

Building Container Images in Docker" ko **deeply, step-by-step**, bilkul real-world example ke saath — taaki tum industry-level confidence ke saath samajh jao.

📦 What is a Docker Image?

Docker image ek **blueprint (design)** hoti hai jo batata hai ki container ko kaise banaya jaye aur usme kya-kya ho.

Yani image = **Operating system + your code + libraries + environment + instructions**

Ye image ke base pe container banaya jata hai.

🏗️ How to Build a Docker Image?

2 Main Methods:

Method	Description
1️⃣ **Using Dockerfile**	Recommended, declarative way
2️⃣ **Using docker commit**	Manual container to image (rare)

We'll focus on **Dockerfile method** (used in industry)

Example Project Folder Structure:

```
myapp/
├── Dockerfile
├── app.js
├── package.json
```

Suppose ye ek Node.js app hai.

Sample Dockerfile:

```
# Step 1: Use base image
FROM node:18
```

```
# Step 2: Set working directory

WORKDIR /app


# Step 3: Copy files to container

COPY package.json .

RUN npm install


# Step 4: Copy rest of the code

COPY . .


# Step 5: Expose port

EXPOSE 3000


# Step 6: Start the app

CMD ["node", "app.js"]
```

Line-by-Line Explanation:

Line	Explanation
`FROM node:18`	Start from official Node.js base image
`WORKDIR /app`	Inside container, make `/app` the current directory
`COPY package.json .`	First copy dependency file
`RUN npm install`	Install required node modules
`COPY . .`	Copy all app files
`EXPOSE 3000`	Inform Docker that app runs on port 3000
`CMD ["node", "app.js"]`	Start the Node.js app when container starts

Build the Image

From the folder where Dockerfile is present:

```
docker build -t mynodeapp:1.0 .
```

Part	Meaning
`-t mynodeapp:1.0`	Tag the image with name + version
`.`	Use current folder as build context

Verify Image

```
docker images
```

Output:

```
REPOSITORY      TAG        IMAGE ID       SIZE

mynodeapp       1.0        234fabcab1      150MB
```

Run a Container from the Image

```
docker run -p 3000:3000 mynodeapp:1.0
```

Yeh command:

- Image se container banata hai
- Port 3000 pe app ko expose karta hai

Build Context – Important Concept

Jab `docker build` chalate ho, Docker poore folder ko (except `.dockerignore`) **"build context"** banata hai — jisme se files container ke andar copy hoti hain.

⚠ Isiliye large file, node_modules, logs ko `.dockerignore` mein daalna chahiye.

Best Practices for Building Images

Best Practice	Why It Matters
Use minimal base images	Reduce image size (like `alpine`)
Use `.dockerignore` file	Speed up builds, avoid junk
Pin image versions	Avoid breaking changes (e.g., `node:18`)
Combine `COPY` and `RUN` steps	To reduce layers & size
Avoid secrets in Dockerfile	Never store passwords or keys there

📊 Industry Examples

App Type	Base Image Used	Registry to Push
Node.js App	`node:18-alpine`	Docker Hub / AWS ECR
Python Flask	`python:3.11-slim`	GitHub Container Registry
Java App	`openjdk:17-jdk-slim`	Azure Container Registry

Summary

Step	Command / Concept
Write Dockerfile	Define image build instructions
Build Image	`docker build -t image:tag .`
Run Container	`docker run -p port:port image:tag`
Push to Registry (optional)	`docker push myrepo/image:tag`
Follow best practices	For secure, small, fast images

Bonus: Multi-Stage Build (Advanced but useful)

For production builds, you can use this to build + deploy in same file without extra weight.

Would you like to learn **Multi-Stage Docker Builds** too?

Question:-

Container Registries?

Answer:-

"Container Registries" ke concept pe — industry mein ye **bahut hi important role** play karta hai CI/CD, DevOps, Kubernetes aur deployment mein.

real-life example, **story style**, aur **industry-level tools** ke saath smjhty hain,

What is a Container Registry?

Container Registry ek **online locker (storage)** hota hai jahan Docker images store hoti hain — taaki tu kisi bhi system/server se us image ko download ("pull") karke use kar sake.

- **Image** = Tiffin box
- **Registry** = Tiffin ka fridge
- **Container** = Plate mein nikaal ke khilaya gaya khaana 🍽️

🏠 Real-World Example:

Step	What Happens
Dev ne image banayi	`docker build -t myapp:1.0 .`
Usne image ko upload kiya	`docker push myregistry/myapp:1.0`
Server ne image pull kiya	`docker run myregistry/myapp:1.0`
App server pe chali	Container ban gaya, production me chali

📦 Types of Container Registries

Public Registries (Free/Open)

Name	Use Case
Docker Hub	Most popular public registry
GitHub Packages	Image storage with repo
GitLab Container Registry	GitLab CI/CD se direct integrate

Private Registries (Secure & Custom)

Name	Company Use
AWS ECR	AWS ka private registry
GCP GCR/Artifact Registry	Google Cloud mein registry
Azure ACR	Microsoft Azure registry
Harbor	Self-hosted enterprise option
JFrog Artifactory	Secure + versioned registry

🛠️ Basic Commands to Work with Registries

1. Build an image

```
docker build -t myapp .
```

☁ 2. Tag the image (before pushing)

```
docker tag myapp docker.io/surajsinha/myapp:1.0
```

📤 3. Login to registry

```
docker login
```

4. Push the image

```
docker push surajsinha/myapp:1.0
```

📥 5. Pull the image (on server)

```
docker pull surajsinha/myapp:1.0
```

Docker Hub vs AWS ECR Example

Feature	Docker Hub	AWS ECR
URL Format	`docker.io/user/image`	`account-id.dkr.ecr.aws.com`
Public/Private	Both	Private (mostly)
Login	`docker login`	`aws ecr get-login-password`
Permissions	Simple	IAM-based security

⚡ Use in CI/CD Pipelines

Task	Tool Example
Build image	Jenkins, GitHub Actions
Push to registry	`docker push`
Kubernetes pull image	Pod pulls image from registry
Image scan	Trivy, Clair, Snyk, Scout

Bonus: Run Your Own Registry (Dev/Testing)

You can even run a local registry on your laptop:

```
docker run -d -p 5000:5000 --name registry registry:2
```

Push to it:

```
docker tag myapp localhost:5000/myapp
```

```
docker push localhost:5000/myapp
```

Pull from it:

```
docker pull localhost:5000/myapp
```

Useful for testing in air-gapped environments or custom setup!

Summary

Term	Meaning
Container Registry	Storage for Docker images
Public Registry	Docker Hub, GitHub Packages
Private Registry	AWS ECR, GCR, Azure ACR
Push	Upload image to registry
Pull	Download image from registry
Real-world importance	CI/CD, Kubernetes, Prod deployments

Want Hands-On?

- Docker image push/pull hands-on chahiye?
- AWS ECR par image upload live example?
- Jenkins + Docker + Registry full CI/CD flow?

Question:-

Docker image Tagging best practices

Answer:-

"Image Tagging Best Practices" hai, to yeh ek bahut important concept hai

samjhte hain **simple story + real-world + hands-on ke saath**, jaise ki mummy ko samjha rahe ho tiffin ke stickers lagane ka

🏷️ What is Image Tagging?

Docker image **tag** = ek naam + version ka sticker
Jisse aap future mein image easily **identify**, **use**, aur **rollback** kar sako.

Real-life example:

- Tiffin box ke upar: `RotiSabzi_Monday` likha ho → pata chal gaya kya aur kab
- Docker image pe: `myapp:1.0.2` → pata chal gaya kya version hai

Format of a Docker Tag

```
docker tag <source-image> <repository>:<tag>
```

Example:

```
docker tag myapp myregistry.com/myapp:1.0.0
```

What If You Don't Tag?

- Docker **default tag** = `latest`
- Lekin **"latest" doesn't mean most recent** – it's just a label!
- Isse confusion hota hai: CI/CD mein kaunsi image chal rahi? Old ya new?

Best Practices for Image Tagging

Best Practice	Why It's Important (Simple Words)
Use Semantic Versioning (`1.0.0`)	Clear version history (major.minor.patch)
Use date-based tags (`2024-06-28`)	Especially for daily releases or auto builds
📦 Use Git commit hash/tag (`v1.0-4g7a`)	Perfect match to your code snapshot
Tag `dev`, `qa`, `prod` images	Separate environments with separate image tracking
Avoid only using `latest`	It's ambiguous and risky for production
Use build number from CI/CD	Unique version per build e.g., `myapp:build-42`

Hands-On: Tagging and Pushing

```
# Build image with tag

docker build -t myapp:1.0.0 .


# Add environment tag

docker tag myapp:1.0.0 myapp:dev


# Push both

docker push mydockerhub/myapp:1.0.0

docker push mydockerhub/myapp:dev
```

Example: Real-Life Project

Imagine you're building an app daily:

Image Tag	Meaning
`myapp:1.0.0`	First release
`myapp:1.0.1`	Bug fix
`myapp:1.1.0`	New feature added
`myapp:2024-06-28`	Daily auto build
`myapp:prod`	Stable prod image
`myapp:commit-abc1234`	Exact match to code version

What Happens If You Don't Follow This?

Mistake	What Can Go Wrong
Using only `latest`	App crashes due to unexpected version
No version control	Can't rollback if bug comes
Same tag for all builds	Difficult to trace which image is live

Industry Tips

Tool	Use
GitHub Actions	Use `GITHUB_SHA`, `GITHUB_REF`, or date in tags

Jenkins	Use `BUILD_NUMBER` or Git hash
ArgoCD/K8s	Always use explicit version tags in YAML files

Summary: Tag Like a Pro

Do This	Don't Do This
`myapp:1.0.1`, `myapp:prod`	Only `myapp:latest`
Tag per environment	Same image for dev & prod
Add Git commit/date	No link to code
Use unique tags in CI/CD	Reuse same tag in every push

Want Hands-On Next?

Tum batao:

- Kya main ek **Jenkins pipeline** bana du jisme har build:
  - Docker image banata hai
  - Usme tag add karta hai (like `myapp:build-42`)
  - Image push karta hai DockerHub ya AWS ECR

Question:-

Running Containers?

Answer:-

next topic: "Running Containers" hai — yeh Docker ka **foundation level** concept hai, bina iske koi DevOps pipeline chalti nahi.

What is a Running Container?

Docker image = **Tiffin Box**

Docker container = **Garama-garam khana plate mein** 🍽

To jab hum `docker run` likhte hain, to hum image ko **real mein chala rahe hote hain** — ek **live process** ban jaata hai system pe.

Command: Run a Container

```
docker run hello-world
```

Yeh kya karega?

- Docker Hub se `hello-world` image pull karega (agar local mein nahi hai)
- Ek container bana kar chalayega
- Screen pe message show karega: "Hello from Docker!"

Real App Example (like Node.js or Nginx)

```
docker run -d -p 8080:80 nginx
```

Part	Meaning
`-d`	Detached mode (background mein chalega)
`-p 8080:80`	Host port 8080 → Container port 80
`nginx`	Nginx image use karo

Ab browser mein `http://localhost:8080` kholke dekho — Nginx chal raha hai!

🏷 Check Running Containers

```
docker ps
```

Output Column	Meaning
CONTAINER ID	Unique ID of running container
IMAGE	Kaunsi image se container bana
PORTS	Kaunsa port map hua hai
STATUS	Running since kitna time

Stop/Start/Remove Containers

Action	Command
Stop	`docker stop <container-id>`
Start again	`docker start <container-id>`
Remove	`docker rm <container-id>`

Remove + force	`docker rm -f <container-id>`

Difference: `docker run` vs `docker start`

`docker run`	`docker start`
New container banata hai	Existing stopped container chalu karta hai
Har baar naya ID milega	Wahi purana container chalu hoga

🛠 Naming Containers (Easier management)

```
docker run -d --name webserver -p 8080:80 nginx

docker ps

docker stop webserver

docker start webserver
```

Hands-on Challenge for You:

1. Run a container with your name as container name:
2. `docker run -d --name suraj-nginx -p 5050:80 nginx`
3.
4. Verify it's running:
5. `docker ps`
6.
7. Stop and remove it:
8. `docker stop suraj-nginx`
9. `docker rm suraj-nginx`
10.

🎓 Real-World DevOps Mein Use

Where Used	Why It’s Important
Jenkins pipeline	Build karne ke baad app run karna
Kubernetes pod backend	Docker containers ke upar hi based hote hain
Local testing/debugging	Developer apna code test karta hai
Monitoring tools	Containers ko observe karte hain

Recap:

Concept	Command Example
Run container	`docker run -d -p 80:80 nginx`
Check running	`docker ps`
Stop container	`docker stop <id or name>`
Remove container	`docker rm <id or name>`
Name container	`--name suraj-nginx`

😎 Ready for the next?

Agle topics ho sakte hain:

- Container logs
- Attach/detach
- Interactive container
- Volumes
- Networking

Question:-

1️⃣ docker run,

2️⃣ docker compose, aur

3️⃣ runtime configuration options

Answert:-

Ab **"Running Containers"** topic ke under ke teen topics:

1️⃣ `docker run`, 2️⃣ `docker compose`, aur 3️⃣ `runtime configuration options`
— in teeno ko ekdum **simple + use-case + hands-on style** mein samajhte hain

1. `docker run` — Core Command

`docker run` = Image se ek **new container banakar** use **chalu (run)** karna

Basic Syntax:

```
docker run [options] image [command]
```

Examples:

🟢 ***Simple Web Server:***

```
docker run -d -p 8080:80 nginx
```

Flag	What it does
`-d`	Detached mode (background)
`-p`	Port mapping (host:container)
`nginx`	Image name

Run with Custom Name:

```
docker run --name myweb -p 5050:80 nginx
```

Run and delete after exit:

```
docker run --rm ubuntu echo "Hello"
```

⚡ Container will auto-delete after running

2. Docker Compose — Multiple Containers Easy

Docker Compose = ek YAML file jisme **multiple containers** ka config hota hai
Jaise **Docker ka orchestra conductor** — sabko ek saath chalata hai

`docker-compose.yml` Example:

```
version: '3'

services:

  web:

    image: nginx
```

```
    ports:
      - "8080:80"
  app:
    image: node:18
    volumes:
      - ./app:/usr/src/app
    command: node server.js
```

Run All:

```
docker-compose up -d
```

📦 Benefits of Docker Compose:

Feature	Why Useful
Multiple services	Run Nginx + Node + DB together
Easy config	Just 1 file for whole setup
Reusable	Use across dev & prod

3. Runtime Configuration Options

Docker run ke samay aap container ka **behavior customize** kar sakte ho.

Important Options:

Option	Example	Use
`--name`	`--name myweb`	Custom container name
`-p`	`-p 8080:80`	Port mapping
`-e`	`-e ENV=production`	Set environment variables
`-v`	`-v /data:/app/data`	Volume mount (host:container)
`--network`	`--network mynet`	Custom Docker network
`--restart`	`--restart unless-stopped`	Auto-restart on failure
`--rm`	`--rm`	Auto-delete after stop
`--cpu` & `--memory`	`--cpus="1.0" --memory="512m"`	Resource limits
`--entrypoint`	`--entrypoint /bin/bash`	Change default command

Full Example:

```
docker run -d \
  --name mynode \
  -p 3000:3000 \
  -e NODE_ENV=production \
  -v $(pwd):/usr/src/app \
  --restart unless-stopped \
  node:18 node server.js
```

This container will:

- Run in background
- Start on boot again if stopped
- Have environment & volume set

Summary Table

Command	Purpose
`docker run`	Run single container
`docker-compose`	Run multiple containers in sync
`runtime options`	Customize container behavior at runtime

Want Practice Task?

Bhai tu try kar:

1. Ek Node.js app banakar Dockerfile likh
2. Fir `docker run` se manually chala
3. Fir `docker-compose.yml` banakar same app aur ek MongoDB chala
4. Use env vars (`-e`) aur volume (`-v`) bhi pass kar

Question:-

Container Security hai Jisme Runtime security and image security

Answer:-

Container Security topic DevOps ka **most critical** aur **interview favorite** subject hai — aur industry mein iska use **daily basis** pe hota hai.

Hum isse **2 major parts** mein todh kar samjhenge:

Part 1: Image Security

Part 2: Runtime Security

easy language + real-world example + tools + commands ke saath smjhty hain.

Part 1: Image Security — "Secure banana chalu hone se pehle"

Why?

Docker image ke andar hi puri app, base OS, aur libraries hoti hain — agar inme vulnerability ho to poori container insecure ban jaata hai.

⚠ Image Security Problems:

Problem	Risk
Outdated base image	Known CVEs (Common Vulnerabilities)
Extra tools in image	Attack surface badhta hai (e.g. curl, nano)
Secrets inside Dockerfile	Passwords leak ho sakti hain
Unverified public images	Malware injected ho sakta hai

Best Practices for Image Security:

Best Practice	Reason
Use minimal base image	Alpine or slim image — kam packages = kam risk
🔒 Never hardcode secrets	Use env vars or secret management tools
Remove unnecessary tools	`curl`, `wget`, etc. dev ke liye hi rakho
Pin image versions	Always use specific tags like `nginx:1.25.2`
🔍 Scan image before use	Use Trivy, Docker Scout, or Snyk

Hands-On: Image Security Scan with Trivy

1. Install Trivy (1-time):

```
sudo apt install wget -y

wget
https://github.com/aquasecurity/trivy/releases/latest/download/trivy_0.48.1_L
inux-64bit.deb

sudo dpkg -i trivy_*.deb
```

1. Scan image:

```
trivy image node:18
```

Output:

- CVEs with severity (HIGH, CRITICAL)
- Affected packages
- Fix recommendations

Dockerfile Secure Example

```
FROM node:18-slim

WORKDIR /app

COPY . .

RUN npm ci --omit=dev && rm -rf /tmp/*

CMD ["node", "server.js"]
```

What’s secure here?

- Slim base image
- No dev packages
- Clean temp folder
- No secrets

🛡️ Part 2: Runtime Security — "Container chalu hone ke baad ki suraksha"

Why?

Even if image secure hai, running container pe bhi attacks ho sakte hain — e.g. shell access, privilege escalation, container escape.

⚠ Runtime Security Risks:

Risk	Description
Root access in container	Poora host OS control le sakta hai attacker
Open ports	Attack entry points
Excess resource usage	Denial of Service (DoS) ho sakta hai
No logging/monitoring	Hack hone ke baad pata bhi nahi chalta

Runtime Security Best Practices:

Practice	Benefit
Never run as root	Use `USER` in Dockerfile
📊 Monitor container behavior	Use Falco, Sysdig, Prometheus
Restrict capabilities	Disable Linux capabilities not needed
Use read-only file system	Prevent code modifications at runtime
Limit open ports	Only expose what’s needed
⚙ Use cgroups & namespaces	Isolate resources (CPU, memory, etc.)

Hands-On: Run Secure Container

```
docker run -d \
  --read-only \
  --cap-drop ALL \
  --pids-limit 100 \
  --memory="512m" \
  --cpus="1.0" \
  --name secureapp \
```

```
node:18-slim node server.js
```

Yeh kya kar raha hai?

- `--read-only` = file system ko likhne se rokta hai
- `--cap-drop ALL` = unnecessary privileges hata deta hai
- `--pids-limit` = zombie processes se bachaav
- `--memory` & `--cpus` = DoS se protection

🛡️ Tools for Runtime Security

Tool	Use
🔍 Falco	Detect suspicious behavior (real-time alerts)
📊 Sysdig	Deep container activity analysis
📉 Prometheus	Metrics + Alerts for CPU/Mem/network
AppArmor / SELinux	Kernel-level isolation & protection

📦 Summary Table

Area	Image Security 🔒	Runtime Security 🛡️
Base image	Use slim, updated	N/A
Secrets	Use env vars, never hardcode	N/A
User access	N/A	Never run as root
Monitoring	Scan with Trivy, Snyk, Scout	Use Falco, Sysdig, Prometheus
Capabilities	N/A	Drop ALL, restrict PIDs & memory

Question:-

Docker CLI -> Image , Containers, Volume, Network

Answer:-

Docker CLI (Command Line Interface) ko samjhte hain **use-cases ke saath, practical style mein** — taki tum har command **na sirf yaad karo, balki use bhi kar pao**

What is Docker CLI?

Docker CLI = ye wo tool hai jise hum terminal/command prompt mai Docker se baat karte hain. Jaise: `docker run`, `docker ps`, `docker build` — sab CLI ke commands hain.

Ye command-line tool Docker daemon ko instructions bhejta hai:

- container chalao
- image banao
- network banao
- volume delete karo
  ...sab kuch!

CLI Ka Format:

```
docker <command> [options]
```

Example:

```
docker run -d -p 80:80 nginx
```

🗂 Docker CLI Commands – Categories ke Saath

Category	Common Commands	What They Do
Images	`docker build`, `docker pull`, `docker images`, `docker rmi`	Build, download, list, delete images
Containers	`docker run`, `docker ps`, `docker stop`, `docker rm`, `docker exec`, `docker logs`	Start, stop, view, delete, debug containers
Volumes	`docker volume create`, `ls`, `inspect`, `rm`	Store persistent data across container restarts
Networks	`docker network create`, `ls`, `inspect`, `rm`	Isolate or connect containers using bridge or custom networks
📦 **Registries**	`docker login`, `docker tag`, `docker push`, `docker pull`	Interact with Docker Hub or AWS ECR
⚙ **System**	`docker info`, `docker version`, `docker system prune`	Docker installation details and cleanup

Most Useful Docker CLI Commands with Example

1. Image-Related:

```
docker build -t myapp:1.0 .
```

Build image from Dockerfile.

```
docker images
```

List all downloaded images.

```
docker pull nginx
```

Download image from Docker Hub.

```
docker rmi nginx
```

Remove a local image.

2. Container-Related:

```
docker run -d --name web -p 8080:80 nginx
```

Start a container named `web` in background.

```
docker ps -a
```

List running + stopped containers.

```
docker stop web
```

```
docker start web
```

Stop/start a container by name or ID.

```
docker logs web
```

See container's log output.

```
docker exec -it web bash
```

Run a command **inside** running container (`bash shell`).

📦 3. Volume Commands:

```
docker volume create myvol
```

```
docker run -v myvol:/data nginx
```

Create and mount a volume for persistent data.

4. Network Commands:

```
docker network create mynet
```

```
docker network ls
```

Create or list Docker networks (used for multi-container apps).

5. Registry Commands (DockerHub or AWS ECR)

```
docker login
```

```
docker tag myapp:1.0 username/myapp:1.0
```

```
docker push username/myapp:1.0
```

```
docker pull username/myapp:1.0
```

Tag, push and pull images to/from registry.

6. Clean-up Commands:

```
docker system prune -a
```

Clean all unused containers, images, networks.

```
docker volume rm myvol
```

```
docker network rm mynet
```

Delete volume/network.

Practice Task for You:

1. Build an image:

```
docker build -t myhello:1.0 .
```

1. Run a container:

```
docker run -d --name hello-container -p 8080:80 myhello:1.0
```

1. Watch logs:

```
docker logs hello-container
```

1. Stop and remove:

```
docker stop hello-container
docker rm hello-container
```

Want CLI Cheatsheet?

Part 1: Docker Volumes (Data Persistence)

📦 Imagine:

Tum ek **hotel** mein ho. Har baar guest aata hai (container), use ek **locker (volume)** diya jaata hai jisme uska saman (data) rakha jata hai.
Container chale ya band ho — **locker safe rehta hai.** 🔒

Why We Need Volumes?

Situation	Problem Without Volume
Container restarts	Data chala jaata hai
Multiple containers share data	Hard without common folder
Need backup / restore	No persistent location

Volume Advantages

- **Persistent data** even if container dies
- Shareable between containers
- Better for backups
- Easier than `bind mounts` (host path)

Volume Commands

```
# Create a volume

docker volume create mydata


# List volumes

docker volume ls


# Inspect volume details

docker volume inspect mydata


# Remove volume

docker volume rm mydata
```

Use Volume in Container

```
docker run -d \

  --name web \

  -v mydata:/usr/share/nginx/html \

  nginx
```

➡ Jo bhi Nginx serve karega, wo ab volume `mydata` ke content se hoga.
Volume ka data safe hai, container delete hone par bhi.

Try This:

1. Create a volume:

```
docker volume create surajvol
```

1. Run a container and mount:

```
docker run -dit --name mybox -v surajvol:/data ubuntu
```

1. Enter and write data:

```
docker exec -it mybox bash
```

```
echo "hello suraj" > /data/notes.txt
```

1. Delete container and re-run:

```
docker rm -f mybox
```

```
docker run -it --name testbox -v surajvol:/data ubuntu cat /data/notes.txt
```

Data still exists! **This is Volume Magic!** 🪄

Part 2: Docker Networks (Container Communication)

Imagine:

Containers = **Rooms**
Docker network = **Internal intercom line**
Public port = **Reception phone**

Why Docker Network?

Goal	How Network Helps
App talks to DB	Same bridge or custom network
Avoid conflict in prod	Isolated networks
External access to app	Port binding from container to host
Secure internal-only services	Use internal-only networks (no host bind)

Types of Docker Networks

Type	Description
bridge	Default. Containers can talk to each other on same network
host	Uses host's network stack (fast but insecure)
none	No network (fully isolated)
overlay	For multi-host (Swarm or K8s setup)

Network Commands

```
# List all networks

docker network ls


# Create custom bridge

docker network create mynet


# Inspect network

docker network inspect mynet


# Remove network

docker network rm mynet
```

Hands-On: Custom Network with App + DB

1. Create a network:

```
docker network create appnet
```

1. Run DB container:

```
docker run -d --name mydb --network appnet mongo
```

1. Run app container:

```
docker run -it --rm --network appnet mongo mongo --host mydb
```

Yeh `--network` use karke tumhare app aur db same virtual LAN pe hain.

Important: Container Name = DNS

Docker networks mein **container ka name = host name** hota hai.
So `myapp` can reach `mydb` by name automatically. No need for IPs.

Real-World DevOps Use

Use Case	Docker Feature Used
Persistent logs	Volumes
MongoDB or MySQL data	Volumes
Node app ↔ MongoDB connection	Custom bridge network
CI/CD testing isolated apps	Custom networks
Sharing env configs	Shared volumes

Bonus: Combine Volume + Network (Real App)

```
docker network create webnet
```

```
docker volume create webdata
```

```
docker run -d \
  --name web \
  --network webnet \
  -v webdata:/usr/share/nginx/html \
```

```
nginx
```

Summary Table:

Feature	Docker Volumes	Docker Networks
Purpose	Persistent data storage	Container-to-container communication
Key command	`docker volume`	`docker network`
Default	Anonymous volume	`bridge`
Custom use	Backup, shared logs, DB storage	App ↔ DB, API ↔ Worker setup

Question:-

Deploying Containers

- **PaaS Option**
- **Kubernetes**
- **Docker Swarm**
- **Nomad**

Answer:-

Ab chalte hain **DevOps journey ke ek super important padav** par:

➤ Deploying Containers — real-world level pe, step-by-step

What Does “Deploying Containers” Mean?

"Container build hone ke baad, usse kisi server par run karna jahan se **users use access** kar saken — isse hi deployment kehte hain."

Ye **local dev** se nikal ke **real production** tak le jaane ka process hota hai.

Container Deployment Options – Real-World Style:

Deployment Option	Where It's Used	Example Commands / Tools
Localhost (Dev)	Testing, Debugging	`docker run`, `docker-compose up`
Cloud VM (EC2)	Lightweight production apps	`scp`, `docker run` on EC2
🕸 Web Server (Nginx)	Host container via reverse proxy	Docker + Nginx config
Kubernetes	Enterprise-scale orchestration	`kubectl apply`, Helm charts
CI/CD pipeline	Auto deployment after commit	GitHub Actions, Jenkins
PaaS	Simplified container platforms	Heroku, Render, Railway, ECS Fargate

Step-by-Step Breakdown of Container Deployment Process

1. Build Your Container Image

```
docker build -t myapp:v1 .
```

Tumne Dockerfile banaya, ab image ban gayi

2. Run Locally First (Verify)

```
docker run -d -p 3000:3000 myapp:v1
```

- `-p` maps container port → host port
- App ab `localhost:3000` pe chal rahi hai

3. Push Image to Container Registry

```
docker tag myapp:v1 suraj/myapp:v1

docker login

docker push suraj/myapp:v1
```

Isse tumhari image Docker Hub ya AWS ECR mein chali jaati hai

4. Deploy to Server (e.g., AWS EC2)

```
# SSH into EC2
```

```
ssh ec2-user@<EC2-IP>


# Pull image
docker pull suraj/myapp:v1


# Run it
docker run -d -p 80:3000 suraj/myapp:v1
```

Ab tumhari containerized app public IP pe accessible hai!

Optional: Reverse Proxy (Nginx)

```
server {
  listen 80;
  server_name yourdomain.com;


  location / {
    proxy_pass http://localhost:3000;
  }
}
```

Nginx forwards domain traffic to Docker container

5. CI/CD Based Deployment

After every Git push → app auto build + test + deploy ho

Example (GitHub Actions):

```
on: [push]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
```

```
      - uses: actions/checkout@v4
      - run: docker build -t myapp .
      - run: docker push myapp
      - run: ssh ec2-user@host "docker pull myapp && docker restart"
```

☸ 6. Production Scale: Kubernetes (K8s)

Kubernetes helps in **automating** container deployment, scaling & management.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: suraj/myapp:v1
          ports:
            - containerPort: 3000
kubectl apply -f deployment.yaml
```

Real-World Example Flow (Mini Project Style)

```
# 1. Build the image

docker build -t suraj/demoapp .


# 2. Run locally

docker run -d -p 3000:3000 suraj/demoapp


# 3. Push to registry

docker push suraj/demoapp


# 4. Deploy to server (EC2)

ssh ec2-user@<public-ip>

docker run -d -p 80:3000 suraj/demoapp
```

➡ App is now live at: `http://<EC2-PUBLIC-IP>`

🛡 Bonus: Deployment Best Practices

Practice	Why?
Use ENV vars for config	Never hardcode secrets
Multi-stage Docker builds	Reduce image size
Use Docker healthchecks	Auto-recovery if app fails
CI/CD integration	Auto test + deploy flow
Monitor containers	Use Prometheus, Grafana

PaaS Option:-

Ab chalte hain **Deploying Containers** ke pehle sub-topic pe:

PaaS Option — What is it, why it's used, and how DevOps mein kaam aata hai.

What is PaaS?

PaaS = Platform as a Service

Ye ek **cloud service model** hota hai jisme tumhe infrastructure (server, network, OS) manage nahi karna padta — tum **bas apni app deploy karo**, aur sab kuch backend pe cloud provider sambhalta hai.

Example socho:

Tum ek chai shop chala rahe ho:

- **IaaS (Infrastructure as a Service)**: Tum khud chai banate ho, gas lagate ho, cup dhote ho
- **PaaS**: Tumhe bas chai banani hai, sab saman (stove, cup, pani) milta hai ready
- **SaaS**: Tum ready-made chai lete ho

Why Use PaaS for Containers?

Benefit	Explanation
No infra management	Server, OS, Docker install — sab handled
Auto-scaling	Load badhne pe container instances auto grow
CI/CD integrated	Code push karo → auto deploy
Developer-friendly	Logs, metrics, dashboard — sab GUI se

Real-World PaaS Platforms (Used with Containers)

PaaS Name	Container Support	Notes
Heroku	Yes (via `Dockerfile`)	Super easy for Node, Python
Railway	Yes	Minimal config, great for prototyping
Render	Yes	Deploy from GitHub in one click
Fly.io	Yes	Global deployment
Google App	Yes	GCP ka PaaS platform

Engine		
AWS App Runner	Yes	Serverless container PaaS by AWS
Azure Web Apps	Yes	PaaS for web containers

Example: Deploy Docker App on Heroku

1. Install Heroku CLI:

```
curl https://cli-assets.heroku.com/install.sh | sh
```

1. Create a Heroku app:

```
heroku login

heroku container:login

heroku create suraj-demo-app
```

1. Push Docker image:

```
heroku container:push web -a suraj-demo-app

heroku container:release web -a suraj-demo-app
```

1. Open the app:

```
heroku open -a suraj-demo-app
```

Boom! App live without worrying about servers

When Should a DevOps Engineer Use PaaS?

Situation	Use PaaS?
Startup or prototype project	Yes
Fast MVP launch without ops team	Yes
Full infra control & tuning needed	× No
Running microservices at scale	× Use Kubernetes (not PaaS)

Limitations of PaaS

✕ Limitation	⚠ Why it matters

Less control on infra	Can't tune OS-level configs
Sometimes costly at scale	Per container cost can increase
Platform lock-in	You get tied to their way
Limited networking/customization	Not for complex infra setups

📦 Summary Table

Feature	PaaS Option
Control	Low (Managed by platform)
Setup Time	Super fast
Suitable For	Devs, fast prototyping, startups
Common Tools	Heroku, Render, Railway, Fly.io, App Runner
DevOps Role	Integrate Docker + CI/CD + ENV config

Docker Swarm:-

important container orchestration tool

Docker Swarm – Container Deployment & Orchestration Tool

What is Docker Swarm?

Docker Swarm ek **native clustering and orchestration tool** hai jo Docker ke andar hi built-in aata hai.
Iska kaam hai **multiple containers** ko across **multiple machines (nodes)** deploy, manage, scale, aur monitor karna — **Docker ke commands se hi**.

Yeh Kubernetes ka ek **simple alternative** hai.

5th Class Example:

Socho tumhare paas 5 dost hain (servers/machines).
Har kisi pe Docker install hai.

Tum chahte ho:

- "Ek app 3 container mein run ho"
- "Ek machine down ho jaye to app phir bhi chale"
- "Traffic auto balance ho"

➡ Docker Swarm aake bolta hai:

“Tension mat le bhai, tu bas `docker service` bol — baaki main handle karta hoon!”

⚙ Docker Swarm Key Concepts

Concept	Meaning (Simple)
Node	Ek machine jisme Docker install hai
Manager Node	Master machine — sab ko manage karta hai
Worker Node	Worker machine — containers run karta hai
Service	Ek app ka instruction: "kitne container run karo"
Task	Ek container running instance inside Swarm
Overlay Network	Multiple node ke containers ko connect karta hai

How Docker Swarm Works

1. Initialize Swarm

```
docker swarm init
```

→ Yeh tumhare system ko **Manager Node** bana deta hai.

2. Add More Nodes (optional)

```
# On worker machines:

docker swarm join --token <worker-token> <manager-ip>:2377
```

→ Tumhare paas ab ek cluster ban gaya hai.

3. Deploy a Service

```
docker service create --replicas 3 --name webapp -p 80:80 nginx
```

✔ Yeh command 3 containers run kar dega — **Swarm khud decide karega** kahan kaunsa chale.

🔍 Useful Commands

```
docker service ls                  # All services

docker service ps webapp          # All tasks (containers) of service

docker node ls                    # All nodes in swarm

docker service scale webapp=5     # Scale container count

docker service rm webapp          # Delete the service
```

Why Use Docker Swarm?

Feature	Benefit
Native to Docker	No need to install extra software
Simpler than K8s	Quick setup and learning
Load balancing built-in	Automatically distributes traffic
Auto healing	Failed containers auto-restarted
Overlay Networking	Connect containers across nodes

📦 Real-World Use Case

Tumhare paas ek project hai jisme:

- 3 containers chahiye frontend ke
- 2 backend containers
- Load balance bhi ho
- 2 VMs hai (Node1, Node2)

Docker Swarm use karke:

- Tum 1 command se deploy kar sakte ho
- Sab automatically spread ho jaayega
- Agar Node2 down ho jaye, toh app phir bhi chalega

Docker Swarm vs Kubernetes (Quick View)

Feature	Docker Swarm	Kubernetes
Complexity	Simple	Complex but powerful
Setup time	Fast (minutes)	Takes more time

Built-in in Docker	Yes	× Needs external setup
Used in prod	Small/medium apps	Large-scale infra
Auto healing	Yes	Yes

Example: Multi-container Service

```
docker service create \
  --name myapp \
  --replicas 3 \
  --publish 8080:80 \
  suraj/myapp:latest
```

- App will run in 3 instances
- Accessible on port 8080
- Auto restarts if crash
- Load balanced

When to Use Docker Swarm?

Use Case	Use Swarm?
Small team, simple app	Yes
Need fast setup	Yes
Already using Docker	Yes
Want deep control/custom infra	× Use K8s
Need third-party integrations	× Use K8s

Docker Swarm in Detail:

Ab main tumhe **Docker Swarm** ka **full working + all commands** samjhata hoon **real-world DevOps project style** mein — jaise tum industry mein kaam kar rahe ho

Docker Swarm: Full Hands-on with Explanation

Objective:

Ek chhoti **3-tier app** (frontend + backend + database) ko **Docker Swarm cluster** pe deploy karna using proper commands.

Step 1: Swarm Cluster Initialize (Single Node First)

```
docker swarm init
```

Ye tumhare system ko **Swarm Manager Node** bana dega.

```
# Output:
```

```
Swarm initialized: current node (abc123) is now a manager.
```

```
To add a worker to this swarm, run the following command:
```

```
docker swarm join --token <token> <manager-ip>:2377
```

Future mein multi-node setup karoge to ye token use hoga

Step 2: Check Cluster State

```
docker info
```

- Check `Swarm: active`
- Check `NodeID`, `Manager` etc.

```
docker node ls
```

- Check all nodes in the swarm (1 for now)

Step 3: Create Overlay Network

Ye **multi-container networking** ke liye zaruri hai

```
docker network create \
```

```
  --driver overlay \
  --attachable \
  my_overlay
```

📦 Step 4: Create Services (Instead of `docker run`)

1️⃣ Frontend (Example: React App)

```
docker service create \
  --name frontend \
  --replicas 2 \
  --network my_overlay \
  -p 80:80 \
  suraj/frontend:latest
```

2️⃣ Backend (Example: Node.js API)

```
docker service create \
  --name backend \
  --replicas 2 \
  --network my_overlay \
  suraj/backend:latest
```

3️⃣ Database (MySQL)

```
docker service create \
  --name db \
  --network my_overlay \
  --env MYSQL_ROOT_PASSWORD=secret \
  mysql:5.7
```

🔍 Step 5: Monitor Your Services

```
docker service ls          # List all services
docker service ps frontend # Show running tasks for frontend
docker service inspect db  # Detailed info
```

Step 6: Scale Services

```
docker service scale frontend=4
```

→ Ab 4 containers run honge frontend ke liye (auto distributed)

⚠ Step 7: Update the Image (Rolling Update)

```
docker service update \
  --image suraj/frontend:v2 \
  frontend
```

→ Old containers ko step-by-step replace karega without downtime

Step 8: Remove a Service

```
docker service rm frontend
```

📦 BONUS: Use `docker-compose.yml` with Swarm

Create a file: `docker-stack.yml`

```
version: "3.8"

services:
  web:
    image: suraj/frontend:latest
```

```
    ports:
      - "80:80"
    networks:
      - mynet
    deploy:
      replicas: 2

  api:
    image: suraj/backend:latest
    networks:
      - mynet
    deploy:
      replicas: 2

  db:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: secret
    networks:
      - mynet

networks:
  mynet:
    driver: overlay
```

Deploy whole stack:

```
docker stack deploy -c docker-stack.yml mystack
```

Sab kuch ek command mein deploy ho gaya!

🔍 Check Stack Status

```
docker stack ls

docker stack services mystack

docker stack ps mystack
```

Remove Stack

```
docker stack rm mystack
```

Bonus Commands

Command	Explanation
`docker node ls`	All nodes in the cluster
`docker node update --availability drain <node>`	Stop scheduling on a node
`docker network ls`	See overlay and bridge networks
`docker service logs <name>`	Show logs of a service
`docker swarm leave --force`	Leave swarm (manager)

Summary

Concept	Command Example
Init Swarm	`docker swarm init`
Create Service	`docker service create ...`
Scale Service	`docker service scale svc=3`
Update Image	`docker service update --image ...`
Full Stack	`docker stack deploy -c stack.yml name`
Remove Service	`docker service rm svc`

Nomad:-

Ab hum **Deploying Containers** ke ek aur advanced aur industry-used orchestration tool pe aate hain:

Nomad by HashiCorp – Container Orchestration Tool

What is Nomad?

Nomad ek **lightweight, flexible, and easy-to-use orchestration tool** hai, jo **Docker containers** ke saath-saath **VMs, binaries, JARs, and QEMU images** bhi run kar sakta hai.
Yeh **HashiCorp** ka product hai (Terraform wale log) — DevOps ecosystem ke liye banaya gaya.

Why Nomad? (Kubernetes ya Swarm kyun nahi?)

Feature	Nomad Speciality
Lightweight	Ek single binary mein pura cluster ready ho jata hai
Multi-runtime support	Docker ke alawa bhi binaries, JARs, VMs run kar sakta hai
Simpler than K8s	Kubernetes jitna complex nahi hai
Integrates with Consul	Service discovery ke liye built-in integration
Integrates with Vault	Secrets management with security

Nomad Architecture

Component	Role
Client Node	Jahan actual workload run hota hai (containers, binaries)
Server Node	Scheduling, cluster coordination, health checking
Jobs	Tumhara application definition (YAML ki jagah HCL file)
Task	Ek job ke andar ke group of containers

Groups

Tasks Individual containers ya binaries jo run hote hain

Nomad vs Kubernetes vs Docker Swarm

Feature	Nomad	Kubernetes	Docker Swarm
Setup complexity	Very low (1 binary)	High	Medium
Built-in discovery	No (uses Consul)	Yes	Yes
Secrets integration	With Vault	With K8s Secrets	Not strong
Production-ready	Yes	Yes	× Not actively developed
Run binaries	Yes	× No	× No
Learning curve	Easy	Steep	Easy

Example Job File – Docker App

```
# web.nomad

job "web" {

  datacenters = ["dc1"]

  type = "service"


  group "web-group" {

    count = 2


    task "web-task" {

      driver = "docker"


      config {

        image = "nginx:latest"

        port_map {

          http = 80

        }

      }
```

```
      resources {
        cpu    = 500
        memory = 256
        network {
          port "http" {}
        }
      }
    }
  }
}
```

Step-by-Step: Nomad Hands-On (Single Node)

Step 1: Install Nomad

```
wget https://releases.hashicorp.com/nomad/1.6.1/nomad_1.6.1_linux_amd64.zip
unzip nomad_1.6.1_linux_amd64.zip
sudo mv nomad /usr/local/bin/
```

Step 2: Start Dev Mode

```
nomad agent -dev
```

Local testing ke liye Dev mode – no cluster config needed

Step 3: Run Job

```
nomad run web.nomad
```

Step 4: Check Job Status

```
nomad status
nomad status web
```

Step 5: Logs

```
nomad logs -job web
```

Access Dashboard

Nomad Dev UI:
http://localhost:4646

Secrets + Service Discovery (Bonus)

Tool	Use with Nomad
Vault	Secrets injection
Consul	Service registration/discovery
Fabio	Load balancing with Consul

Real-world Use Case (DevOps Team)

Team Need	Nomad Solution
Run Go binary + container in one job	Supported via mixed-task group
Manage secrets securely	Vault integration
Auto scaling + job restart	Built-in restart and health check rules
CI/CD with GitHub Actions or Jenkins	Use Nomad CLI in pipeline